

database design

Below is a complete database design specification for NCPS, written so an AI agent or engineer can directly implement it. This includes:

- database selection rationale
- schema (tables + fields + types)
- indexing strategy
- relationships
- hybrid storage design (SQL + cache + graph)

NCPS Database Architecture Design

1. Database Choice (Critical Decision)

Recommended Stack

Primary DB → PostgreSQL Cache Layer → Redis Graph Storage → PostgreSQL (adjacency table) OR Neo4j (optional) Stream Queue → Kafka / RabbitMQ

1.1 Why PostgreSQL (Primary)

PostgreSQL is chosen because:

- strong relational integrity (users ↔ posts ↔ interactions)
- supports JSON for flexible ML metadata
- supports indexing + analytical queries
- handles transactional updates reliably

1.2 Why Redis (Cache Layer)

Used for:

- real-time values (C_j, w_i, Prox, alerts)
- rate limiting
- session and hot data

1.3 Graph Storage Choice

Option A (Recommended for prototype)

PostgreSQL adjacency table

Option B (Advanced)

Neo4j (if graph grows large and complex)

2. Data Model Overview

Users ↓ Interactions → Posts ↓ ↓ Graph Credibility ↓ ↓ Trust Alerts

3. Core Tables

3.1 Users Table

```
CREATE TABLE users ( user_id UUID PRIMARY KEY, -- Reliability alpha FLOAT DEFAULT 0, beta FLOAT
DEFAULT 0, r_score FLOAT, confidence FLOAT, r_star FLOAT, -- Experience exp_raw FLOAT, exp_score FLOAT,
-- Anomaly anomaly_score FLOAT, -- Graph trust trust_score FLOAT, -- Location lat DOUBLE PRECISION, lon
DOUBLE PRECISION, location_confidence FLOAT, -- Metadata created_at TIMESTAMP, updated_at
TIMESTAMP );
```

Indexes

```
CREATE INDEX idx_users_trust ON users(trust_score); CREATE INDEX idx_users_location ON users(lat, lon);
```

3.2 Posts Table

```
CREATE TABLE posts ( post_id UUID PRIMARY KEY, user_id UUID REFERENCES users(user_id), content
TEXT, embedding JSONB, -- Credibility c_bayes FLOAT, c_ml FLOAT, c_memory FLOAT, c_final FLOAT, --
Stability variance FLOAT, -- Interaction mass n_effective FLOAT, -- Urgency urgency FLOAT, -- Propagation
radius FLOAT, -- Timing created_at TIMESTAMP, updated_at TIMESTAMP );
```

Indexes

```
CREATE INDEX idx_posts_credibility ON posts(c_final); CREATE INDEX idx_posts_created ON
posts(created_at);
```

3.3 Interactions Table

```
CREATE TABLE interactions ( interaction_id UUID PRIMARY KEY, user_id UUID REFERENCES users(user_id),  
post_id UUID REFERENCES posts(post_id), vote SMALLINT, – +1 or -1 weight FLOAT, timestamp TIMESTAMP  
);
```

Indexes

```
CREATE INDEX idx_interactions_post ON interactions(post_id); CREATE INDEX idx_interactions_user ON  
interactions(user_id); CREATE INDEX idx_interactions_time ON interactions(timestamp);
```

3.4 User Location History Table

```
CREATE TABLE user_locations ( id SERIAL PRIMARY KEY, user_id UUID REFERENCES users(user_id), lat  
DOUBLE PRECISION, lon DOUBLE PRECISION, timestamp TIMESTAMP );
```

Purpose

- compute movement plausibility
 - compute spatial variance
-

3.5 Graph Adjacency Table

```
CREATE TABLE user_graph ( user_id UUID, neighbor_id UUID, agreement_score FLOAT, time_similarity  
FLOAT, frequency_score FLOAT, edge_weight FLOAT, last_updated TIMESTAMP, PRIMARY KEY (user_id,  
neighbor_id) );
```

Index

```
CREATE INDEX idx_graph_user ON user_graph(user_id);
```

3.6 Alerts Table

```
CREATE TABLE alerts ( alert_id UUID PRIMARY KEY, user_id UUID, post_id UUID, timestamp TIMESTAMP,  
delivered BOOLEAN DEFAULT FALSE );
```

Index

```
CREATE INDEX idx_alert_user_time ON alerts(user_id, timestamp);
```

3.7 Rate Limiting Table (Optional)

CREATE TABLE user_alert_limits (user_id UUID PRIMARY KEY, alert_count INT, last_reset TIMESTAMP);

4. Derived / Computed Values (Not Stored Permanently)

These should be computed dynamically or cached:

- $\text{Prox}(u, j, t)$
- $\text{Burst}_j(t)$
- $V_j(t)$
- $C_{\{\text{memory}\}}(j, t)$

5. Redis Cache Design

5.1 Cached Keys

user:{id}:weight $\rightarrow w_i$ user:{id}:trust $\rightarrow T_i$ post:{id}:credibility $\rightarrow C_j$ post:{id}:urgency $\rightarrow U_j$ post:{id}:variance $\rightarrow \text{Var}_j$

5.2 Rate Limiting

alert_count:{user_id}

TTL:

60 seconds or sliding window

6. Graph Storage Strategy

Option A: PostgreSQL (Default)

- store adjacency in user_graph
- recompute periodically

Option B: Neo4j (Advanced)

(User)-[SIMILAR {weight}]->(User)

Used if:

- graph size > millions of edges
- need fast traversal

7. Data Flow Mapping

Vote Event

Insert → interactions Update → posts (C_j, N_j) Update → users (alpha, beta, Exp) Update → graph edges
Cache → Redis

Location Update

Insert → user_locations Update → users.location_confidence

Alert Trigger

Insert → alerts Update → Redis rate limit

8. Partitioning Strategy (Scalability)

Posts

PARTITION BY RANGE (created_at)

Interactions

PARTITION BY HASH (post_id)

9. Data Consistency Strategy

Strong Consistency

- users table
 - posts table
-

Eventual Consistency

- graph updates
 - ML outputs
-

10. Minimal Schema Summary

Required Tables

users posts interactions user_locations user_graph alerts

11. Final Recommendation

For Your Prototype

Use:

PostgreSQL + Redis

Avoid Neo4j initially unless:

- graph becomes bottleneck
 - need deep graph traversal
-

12. Final Insight

Your database design reflects your system philosophy:

- users store trust
 - posts store belief
 - interactions store evidence
 - graph stores relationships
 - cache stores real-time state
-